

Graph-Structure-Based Grouping of Web Browser Tabs into Tasks

Mizuki Hisatomi
Graduate School of Informatics
Kyoto University
Kyoto, Japan
hisatomi@dl.soc.i.kyoto-u.ac.jp

Keishi Tajima
Graduate School of Informatics
Kyoto University
Kyoto, Japan
tajima@i.kyoto-u.ac.jp

Abstract

Web browser tabs allow users to work on multiple tasks in parallel. However, when tabs from different tasks are mixed, it becomes difficult to locate the desired tab and maintain focus. Grouping tabs by task can enhance usability, but manual grouping imposes a high cognitive burden, and existing machine learning-based approaches typically require costly manual annotations and rely on privacy-sensitive data such as URLs. In this paper, we propose a method that automatically groups tabs by task without requiring manual annotation or access to sensitive data. Our method builds a directed graph based on tab-switching history and applies a community detection algorithm to identify task-related groups. To evaluate our approach, we conducted a comparative experiment using data collected from eight participants. We compared our method against a variant of an existing machine learning-based approach, modified to operate without private data. Experimental results demonstrate that our method achieves comparable or superior performance without the need for manual annotation.

CCS Concepts

• **Human-centered computing** → **User models**; *Hypertext / hypermedia*; Web-based interaction.

Keywords

tab browsers, multitasking, task-switching, graph partitioning

ACM Reference Format:

Mizuki Hisatomi and Keishi Tajima. 2025. Graph-Structure-Based Grouping of Web Browser Tabs into Tasks. In *The 38th Annual ACM Symposium on User Interface Software and Technology (UIST Adjunct '25)*, September 28–October 01, 2025, Busan, Republic of Korea. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3746058.3758439>

1 Introduction

In web browsers, the ability to open multiple pages simultaneously is essential for multitasking, and most browsers support this via tabbed browsing. However, when too many tabs are open, they become visually cluttered, with tabs from different tasks mixed together. This not only reduces the space available for each tab in the tab bar, making it harder to identify the desired one, but also exposes users to unrelated content, hindering their focus.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST Adjunct '25, Busan, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2036-9/25/09

<https://doi.org/10.1145/3746058.3758439>

To address this, some browsers offer tab grouping functionality, and various extensions assist users in manually organizing tabs [3, 5, 9]. Nonetheless, manual grouping remains time-consuming.

Automatic tab classification methods have been proposed to reduce this burden. Some extensions group tabs by domain name [1, 2], but this approach often fails to reflect actual task boundaries, as the same domain is often involved in multiple tasks. The method proposed in [7] infers task switches from tab transition sequence by using machine learning, but it relies on shared pre-trained models that require access to privacy-sensitive data, such as page titles. While training models locally would address privacy concerns, it still demands substantial manual labeling from users.

In this study, we propose a method that automatically groups browser tabs by task without using privacy-sensitive data or manual annotation data. Our approach constructs a directed graph representing tab transitions and applies a community detection algorithm to cluster related tabs. We evaluated our method by comparing it to a modified version of the approach from [7], adapted to exclude sensitive information. Experimental results show that our method achieves comparable or superior average performance, offering a privacy-preserving solution to the tab classification problem without costly manual annotation.

2 Related Work

The problem of users opening too many tabs in web browsers, reducing usability, has been highlighted in several studies [6, 9, 10]. To mitigate this, many browser extensions have been developed [3, 4]. However, these tools still require manual organization by users.

Several studies also address task management in browsers from other perspectives [9, 11, 12]. These systems, however, also rely on manual classification.

Tabs.do [7] automatically organizes tabs using a neural network. However, it requires access to privacy-sensitive data like URLs and page titles as explained before. In contrast, our study proposes an automatic tab classification method that does not depend on such sensitive information and costly manual annotation data.

3 Proposed Method

Our method classifies tabs by constructing a graph that represents transitions between tabs and applying a community detection algorithm to it. As a pre-processing step, we remove the following two types of accesses from the browsing history: transient accesses and interruptive accesses. Transient accesses are cases where the tab is merely passed through without the user engaging with its contents. For example, this may occur when a user navigates to a target tab using keyboard shortcuts, momentarily accessing intermediate tabs along the way, or when a tab is unintentionally selected during the process of locating the desired tab. Interruptive accesses are

transitions to pages related to other tasks (e.g., checking email or schedules) that occurred during the execution of a different task.

Specifically, we remove a transition as a transient access if either: (1) the dwell time is less than 0.5 seconds, or (2) the dwell time is less than 1 second, and the dwell times of both the preceding and following tabs are also less than 1 second. To identify interruptive accesses, we remove accesses to pages that have transitions with more than 10% of all pages in the user’s browsing history, as these pages are likely to be shared across multiple tasks.

Using the filtered browsing history, we construct a directed graph where each node represents a tab, each edge represents a transition between tabs, and the edge weight corresponds to the number of transitions. However, if the opening times of two tabs differ by more than five minutes, we exclude the edge, as this suggests a possible break in browser usage. While transitions within the same task may still occur after such intervals, shorter transitions are more common within a task. Therefore, we prioritize precision over recall and adopt a conservative threshold of five minutes.

Finally, we apply the directed Louvain method [8] to the resulting transition graph to perform community detection. We regard each detected community as a group of tabs belonging to the same task.

3.1 Baseline Method

For comparative evaluation, we implemented a variant of the previously described Tabs.do [7] approach, modified to avoid the use of privacy-sensitive information. To detect task switches between two consecutive tabs in the browsing history, we used a random forest classifier based on the following features:

- time difference between the opening of the two tabs,
- whether the anonymized URLs of the two tabs are identical,
- whether the anonymized domains of the URLs of the two tabs are identical, and
- number of past transitions between the two tabs.

Using this classifier, we determined whether each pair of consecutive tabs in the browsing history represented a task switch. Tabs between two detected task switches were grouped as a single task segment. After applying this process to the entire browsing history, we merged all tab groups that shared common tabs and treated each resulting merged group as corresponding to a single task.

In our experiments, we trained a model in advance using data from each individual participant, and used the model trained on a specific user’s data to classify that user’s tabs.

4 Experiment

We evaluate the proposed method using three metrics: (1) cost of navigating to other tabs within the same task, (2) purity of tasks in the produced tab groups, and (3) Adjusted Rand Index. The first two are newly defined to assess how well the method addresses our problem, and the third is a standard metric for clustering algorithms.

We first explain the first one. In what follows, we refer to the ground-truth tab groups provided by users as tasks, and the groups produced by each method as generated groups or simply groups. When a task’s tabs are split across multiple groups, the group containing the largest number of that task’s tabs is referred to as the main group for the task. A single group may serve as the main group for multiple tasks.

The navigation cost is defined as the expected number of clicks required to move from one tab in the main group to all other tabs that belong to the same task. For each task, we compute the cost of accessing every other tab in that task, starting from one tab in the main group. The total cost is the sum across all tasks.

The cost is divided into two components: (1) the cost of locating other same-task tabs within the main group, and (2) the cost of locating same-task tabs outside the main group.

Let n and m be the total number of tabs and the total number of generated groups, $C_1, C_2, \dots, C_m$ be the generated groups, $n_i = |C_i|$, t_i be the number of tabs in C_i that belong to the target task, C_{main} be the main group for the target task, n_{main} and t_{main} be the total number of tabs and task-relevant tabs in C_{main} , respectively.

The cost for other same-task tabs within the main group is calculated as follows. Assume a user is viewing tab A of the target task in the main group. To locate another tab B from the same task, the user may sequentially open the remaining tabs. The expected number of clicks required is $n_{\text{main}}/2$. Since this applies to $t_{\text{main}} - 1$ other tabs in the task, the total cost is $(t_{\text{main}} - 1)n_{\text{main}}/2$.

To find a same-task tab in another group C_i , the user must first select the correct group (with one click), and then search within it. If the wrong group is selected, the user must open all its tabs to confirm it is incorrect, but we approximate the number of clicks for it by the average group size: n/m . We omit the details, but the expected number of clicks needed to reach a tab inside some C_i is:

$$\sum_{i=1, \dots, m, C_i \neq C_{\text{main}}} \frac{1}{2} \left\{ m + n_i + 1 + \frac{n(m-2)}{m} \right\} t_i.$$

The sum of these two gives the total expected navigation cost. A lower value indicates a more task-coherent grouping.

We next define task purity. When a group contains many tabs unrelated to the current task, it can distract the user and hinder their ability to concentrate. Task purity is defined as follows:

$$\frac{\sum_{i=1}^m t_i}{\sum_{i \in \{i=1, \dots, m \mid t_i \neq 0\}} n_i}. \quad (1)$$

For a given task, if none of the groups containing its tabs also include tabs from other tasks, the task purity is 1. We compute the average of this value across all tasks. Higher average purity indicates better grouping quality.

Table 1 summarizes the result of our experiment with 15 datasets collected from 8 users. Both methods incorporate the pre-processing steps that remove transient and interruptive accesses. In terms of navigation cost, our method performs comparably to the baseline, while in the other two metrics, it outperforms the baseline. It is worth noting that the baseline requires costly manual task annotations, whereas our method does not. We observed similar results when pre-processing was omitted from both methods.

5 Conclusion

In this paper, we proposed a method for automatically grouping web browser tabs according to the user’s tasks. Our method identifies these groups by applying a community detection algorithm to a transition graph constructed from tab switching history. This approach requires no manual annotation. In our experiment, the proposed method achieved comparable or better performance than a baseline method that relies on costly manual annotations.

Table 1: Comparison of the proposed method and the baseline

user	Nav. cost		Purity		Adjusted Rand	
	prop.	base	prob.	base	prob.	base
D1	305	450	0.319	0.417	0.0607	0.0529
D2	81.6	88.7	0.408	0.595	0.210	0.210
D3	308	171	0.668	0.307	0.0204	0.0570
D4	59.8	60.0	0.568	0.143	0.280	0.00
D5	5.50	7.50	0.633	0.500	0.167	0.00
D6	68.3	87.9	1.00	1.00	0.148	0.0299
D7	776	560	0.736	0.500	0.00673	0.467
D8	145	197	0.712	0.623	0.239	0.122
D9	119	110	0.789	0.620	0.388	0.190
D10	555	417	0.821	0.558	0.0332	-0.0102
D11	378	252	0.750	0.201	0.0145	-0.0618
D12	167	111	0.710	0.250	-0.0152	0.00
D13	48.6	43.8	0.881	0.467	0.118	-0.0377
D14	391	490	1.00	1.00	0.0908	0.0195
D15	7.50	11.6	0.389	1.00	-0.176	0.328

Acknowledgments

The authors would like to thank Takehiro Yamamoto for his valuable advises.

References

- [1] Acid Tabs. <https://chrome.google.com/webstore/detail/acid-tabs/hhgceopemmcmigbmhphbcgkeffommpjfc>.
- [2] Auto-Group Tabs. <https://chrome.google.com/webstore/detail/auto-group-tabs/danneghahnicanipdoajmakdbeaophenb>.
- [3] OneTab. <https://chrome.google.com/webstore/detail/onetab/chphlpgkkbolifaimnlloiipkdnihall>.
- [4] SplitUp! <https://chrome.google.com/webstore/detail/splitup-tab-manager/bhoodecbejheonelhikcfahggahffmf>.
- [5] Tabs Outliner. <https://chrome.google.com/webstore/detail/tabs-outliner/eggkanocgddhmamlbiijnphhpkpkmkl>.
- [6] J.C. Chang, N. Hahn, Y. Kim, J. Coupland, B. Breneisen, H.S. Kim, J.Hwang, and A. Kittur. 2021. When the Tab Comes Due: Challenges in the Cost Structure of Browser Tab Usage. *Proc. of ACM CHI* (2021), 1–15.
- [7] J.C. Chang, Y. Kim, V. Miller, M.X. Liu, B.A. Myers, and A. Kittur. 2021. Tabs.do: Task-Centric Browser Tab Management. *Proc. of ACM UIST* (2021), 663–676.
- [8] N. Dugué and A. Perez. 2022. Direction matters in complex networks: A theoretical and applied study for greedy modularity optimization. *Physica A: Statistical Mechanics and its Applications Vol 603* (2022).
- [9] M. Kulkarni, K. Kapoor, D.S.V Madala, S. Bansal, and S. Hangal. 2019. Compartmentalizing web browsing with Sailboat. *Proc. of Indian Conference on Human-Computer Interaction* (2019), 1–8.
- [10] R. Ma, H. Lassila, L. Nurgalieva, and J. Lindqvist. 2023. When Browsing Gets Cluttered: Exploring and Modeling Interactions of Browsing Clutter, Browsing Habits, and Coping. *Proc. of ACM CHI* (2023), 1–29.
- [11] M. Mayer and B.B. Bederson. 2001. Browsing Icons: A Task-Based Approach for a Visual Web History. *HCIL-2001-19, CS-TR-4308, UMIACS-TR-2001-85, HCI Lab, University of Maryland, Maryland, USA* (2001).
- [12] M.R. Rajamanickam, R. MacKenzie, B. Lam, and T. Su. 2010. A task-focused approach to support sharing and interruption recovery in web browsers. *CHI '10 Extended Abstracts on Human Factors in Computing Systems* (2010), 4345–4350.